innFactory

Anwendung der funktionalen Programmierung mit Scala

TH Rosenheim - SoSe 2025

Makros

innFactory

Makros
Was ist ein Makro?

Makros sind Programme, die **Programmcode generieren**. Sie laufen während der **Kompilierung** und erzeugen Code, der dann zur Laufzeit ausgeführt wird. Dies trennt **Compile-Time** strikt von **Run-Time**.

Makros
Two-Stage-Execution

Ein Makro führt Berechnungen zur **Compile-Time** aus (Stage 1) und baut dabei neuen Code zusammen der resultierende Code wird in Stage 2 (dem laufenden Programm / **Run-Time**) normal ausgeführt.

Dieses *Metaprogramming*-Konzept erlaubt z.B. Konstantenberechnungen oder Boilerplate-Generierung schon beim Kompilieren.

innFactory

Makros
Sprach-Vergleich

In C/C++ kennt man einfache textual Preprocessor-Makros (ohne Typsicherheit), in Lisp Schemes hygienische Makros seit langem. Scala 2 hatte Makros auf Basis von AST-Manipulation, war aber kompliziert und fehleranfällig.

=> Scala 3 führt ein *neues Makro-System* ein, das **typsicher** und **hygienisch** ist. “Hygienisch” bedeutet, dass Makro-Expansionen keine zufälligen Namenskollisionen oder variable Capturing verursachen. Der Compiler sorgt für saubere Trennung der Namensräume und fügt z.B. bei Bedarf automatisch Präfixe ein, um Konflikte zu vermeiden

innFactory

Makros
Anwendungsgebiete

- Generieren von Boilerplate-Code (*Case Class* Serialisierungen / Mapping)
- Konstantenentfaltung (z.B. Berechnung von Tabellen)

=> Dadurch spart man sich wiederholten Code und erhält potentielle Performancegewinne (da Berechnungen vorweggenommen werden)

Achtung: Makros sind zwar mächtig, aber mit Vorsicht einzusetzen. Komplexe Makros können Wartung erschweren (Zu viel “magic”)

innFactory

Quotes & Splices: Makro-Syntax in Scala 3

Quotes & Splices: Makro-Syntax in Scala 3
Quoted Expressions

Scala 3 führt die Syntax '{ ... } ein, um einen **ausdrücklich verschobenen Codeblock** zu markieren. Alles innerhalb von '{ } wird **nicht sofort ausgeführt**, sondern als Daten repräsentiert (man erhält ein Objekt vom Typ Expr[T], das einen Code-AST vom Typ T darstellt).

Man kann sich das wie ein “Code-Template” vorstellen, das noch Lücken enthält.

```
import scala.quoted.{Expr, Quotes}
object Macros {
  def meaningOfLife(using Quotes): Expr[Int] = '{ 42 }
}
```

Quotes & Splices: Makro-Syntax in Scala 3
Splices

Umgekehrt erlaubt ${ ... } das **Einfügen/Splicen** von Code oder Werten in einen quoted Block. Ein Splice innerhalb eines Quotes signalisiert dem Compiler:
*"Führe diesen Part **zur Compile-Time** aus und setze das Ergebnis hier ein."*

Dadurch kann man z.B. berechnete Ausdrücke oder rekursiv generierten Code in einen größeren Ausdruck einfügen.

Wichtig: Ein Splice ${x} kann nur *innerhalb* eines '{ ... } stehen – d.h. man splict in einen Quote-Kontext hinein.

Quotes & Splices: Makro-Syntax in Scala 3

Splices Beispiel

```scala
object Macros {
  def meaningOfLifeImpl(using Quotes): Expr[String] = {
    val num: Expr[Int] = '{ 40 + 2 }
    '{ s"The meaning of life is ${$num}" }
  }
  inline def sayMeaningOfLife: String = ${ meaningOfLifeImpl }

  def main(args: Array[String]): Unit = {
    println(sayMeaningOfLife)
  }
}
```

```
Cannot call macro method meaningOfLifeImpl defined in the same source file
    println(sayMeaningOfLife)
```

Quotes & Splices: Makro-Syntax in Scala 3

Splices Beispiel

```scala
object RunMacro {
  def main(args: Array[String]): Unit = {
    println(Macros.sayMeaningOfLife)
  }
}
```

```
The meaning of life is 42
```

Quotes & Splices: Makro-Syntax in Scala 3
Staging-Regel

Faustregel: Wenn in einem inline-Kontext mehr Splices (${}) als Quotes ('{}) vorkommen, wird der Code als Makro **ausgeführt** (d.h. die überzähligen Splices bewirken Code-Generierung).

Gibt es gleich viele Quotes wie Splices, resultiert ein normaler zur Laufzeit evaluierbarer Ausdruck.

Quotes & Splices: Makro-Syntax in Scala 3
Makro-Expansion

Der Scala-Compiler führt Makros während des Tycheckings aus. Bei einem inline def mit Splice wird die im Splice referenzierte Implementierung aufgerufen, die wiederum Code als Expr[...] zurückgibt. Das Ergebnis ersetzt dann den ursprünglichen Makroaufruf im Code.

Quotes-Context und TASTy-Reflexion

Quotes-Context und TASTy-Reflexion
Quotes-Context(using q: Quotes)

Jede Makro-Implementierung hat (implizit) Zugriff auf ein Objekt quotes: Quotes. Dieses stellt die *Reflection API* bereit (zugänglich über quotes.reflect).

Darüber kann man den AST (Typed Abstract Syntax Tree, *TASTy*) des Codes einsehen und analysieren. Beispielsweise enthält quotes.reflect.Symbol Infos über Bezeichner, quotes.reflect.TypeRepr repräsentiert Typen usw.

Quotes-Context und TASTy-Reflexion
Kontext

```
import scala.quoted.{Expr, Quotes}

object Macros {
  def meaningOfLifeImpl(using q: Quotes): Expr[String] = {
    q.reflect.report.info("Hello from macro!")
    val num: Expr[Int] = '{ 40 + 2 }
    '{ s"The meaning of life is ${$num}" }
  }
  inline def sayMeaningOfLife: String = ${ meaningOfLifeImpl }
}
```

```
object RunMacro {  new *
  def main(args: Array[String]): Unit = {  new *
    println(Macros.sayMeaningOfLife)
  }
}
```

```
Hello from macro!
```

Quotes-Context und TASTy-Reflexion
TASTy-Reflection

Ermöglicht erweiterte Makros, die z.B. durch die Struktur eines Typs iterieren. Etwa um alle Felder einer Case Class zu ermitteln, könnte man über quotes.reflect.Symbol.fields gehen. Dieses niedrige Level ist mächtiger, aber komplexer. AST-Knoten müssen manuell zusammengesetzt werden. Für viele Anwendungsfälle reichen die einfacheren Quote/Splice-Kombinationen *ohne* manuelle Reflection (dazu gleich mehr).

Quotes-Context und TASTy-Reflexion

Inline vs. Reflection-Makros: Scala 3 bietet drei Makro-Ansätze

- *Inline Methoden* (ggf. mit Compile-Time scala.compiletime Ops): Am einfachsten, aber limitiert.
- *Quoted Macros* (Inline + Quotes/Splices): Ermöglicht Code-Generierung mit verhältnismäßig knapper Syntax, für viele Zwecke ausreichend.
- *Full Reflection Macros* (Quotes + quotes.reflect API): Volle Power, um AST nach Belieben zu manipulieren, aber am aufwendigsten.
  => Best Practice: So einfach wie möglich halten. Erst inline verwenden, falls nötig Quotes, und nur wenn unvermeidbar zur Reflection greifen

Makro-Beispiel mkPower

Makro-Beispiel mkPower
Idee

Eine Funktion mkPower(n) soll zur Compile-Time einen Exponenten n nutzen, um eine Potenz-Funktion zu bauen, die zur Laufzeit effizient arbeitet.

```
def mkPower(n: Int): Double ⇒ Double
```

Erwartetes Verhalten: Wenn man z.B. val f = mkPower(3) schreibt, erhält man eine Implementierung mit x => x * x * x. Zur Laufzeit entfällt dadurch jegliche Rechnung außer der Multiplikationen selbst, also z.B. keine Schleife.

Makro-Beispiel mkPower
Umsetzung

mkPower könnte rekursiv etwa wie folgt funktionieren:

- Wenn n == 0: gebe '{ (x: Double) => 1.0 } zurück
- Wenn n == 1: gebe '{ (x: Double) => x } zurück
- Wenn n > 1: generiere '{ (x: Double) => x * ${ ... (n-1 mal) ... } }. Dabei würde man wieder mkPower(n-1) aufrufen und das resultierende Expr in den Splice einbetten.

(Für große n würde der Bytecode entsprechend groß; Makro also nur sinnvoll, wenn n überschaubar ist.)

Makro-Beispiel mkPower
Code

```scala
import scala.quoted.{Expr, Quotes}

inline def mkPower(n: Int): Double ⇒ Double =
  ${ mkPowerImpl('n) }

private def mkPowerImpl(n: Expr[Int])(using q: Quotes): Expr[Double ⇒ Double] = {
  import q.reflect.*
  n.value match {
    case Some(0) ⇒ '{ (x: Double) ⇒ 1.0 }
    case Some(1) ⇒ '{ (x: Double) ⇒ x }
    case Some(k) if k > 1 ⇒
      val prev = mkPowerImpl(Expr(k - 1))
      '{ (x: Double) ⇒ x * ${ prev }.apply(x) }
    case _ ⇒
      report.error("Power must be a non-negative constant integer")
      '{ (x: Double) ⇒ Double.NaN }
  }
}
```

```scala
private val pow2 = macros.mkPower(-3)
```

Power must be a non-negative constant integer

Makros in Production

Makros in Production
Play Framework JSON-Makros (Play-JSON)

- **JSON-Serialisierung in Play:** Zur Konvertierung von Case Classes <-> JSON bietet Play das play.api.libs.json-Modul. Normalerweise müsste man für jede Case Class eine Instanz von Reads (für JSON->Objekt) und Writes (für Objekt->JSON) implementieren. Das ist repetitiv.
- **Automatische Macro-Helfer:** Play stellt Makro-Methoden bereit: z.B. Json.reads[MyClass] generiert einen Reads[MyClass], Json.writes[MyClass] analog für Writes, und Json.format[MyClass] für beides gleichzeitig (ein Format ist Reads+Writes). Diese Makros schauen sich per Reflection/Compile-Time die Case Class an und bauen Code, der die Felder in JSON-Felder mappt.

```scala
case class User(name: String, age: Int)
import play.api.libs.json.*
given Format[User] = Json.format[User]
```

```scala
given Format[User] =
  (__ \ "name").format[String] ~
  (__ \ "age").format[Int] apply User.apply
```

Makros in Production
Chimney: typsichere Objekt-Transformationen

- **Motivation:** Beim Mappen von Daten zwischen ähnlichen Strukturen (z.B. API-DTO zu Domain-Model, oder DB-Entität zu Case Class) schreibt man oft repetitiven Code: target.field1 = source.field1, usw. *Chimney* automatisiert diese **Objekt-Transformationen**.
- **Grundidee:** Chimney stellt eine DSL bereit, man ruft source.into[TargetType] auf und kann dann per Kettensyntax Anpassungen vornehmen, bis man .transform aufruft. Intern nutzt Chimney Makros, um sicherzustellen, dass **alle Felder gemappt** werden und Typen übereinstimmen. Wenn etwas fehlt oder inkonsistent ist, bricht der Compiler mit einer klaren Fehlermeldung ab (typsicher!).

Makros in Production
Chimney Beispiel

Angenommen wir haben case class UserDTO(fullName: String, age: Int) und case class User(name: String, age: Int). Die Feldnamen differieren (fullName vs name). Mit Chimney können wir schreiben:

```scala
import io.scalaland.chimney.dsl.*

val dto = UserDTO("Max Mustermann", 21)
val user: User = dto.into[User]
  .withFieldRenamed(_.fullName, _.name)
  .transform
```

Packaging & Deployment (GraalVM Native Image & Docker)

Packaging
Warum ist das wichtig?

- **Startzeit:** In Cloud- und Serverless-Kontexten sind schnelle Cold-Starts Gold wert: hier kann Packaging (JVM vs. Native) einen Unterschied von Sekunden vs. Millisekunden bedeuten.
- **Ressourcenverbrauch:** Containerisierte Microservices sollten sparsam mit RAM/CPU umgehen. Ein fat JDK-basiertes Image vs. ein schlankes natives Binary-Image kann hunderte MB an Unterschied machen.
- **Deployment & Ops:** Einfache, reproduzierbare Artefakte (Docker-Images, self-contained binaries) erleichtern Continuous Deployment. Zudem sind kleinere Images schneller zu übertragen und sicherer (weniger Angriffsfläche).

Kurz: Packaging ist nicht nur Nebensache des DevOps. Es sollte schon bei der Entwicklung mitgedacht werden.

Packaging
GraalVM Native Image

GraalVM ist eine Technik, um JVM-Anwendungen **Ahead-of-Time (AOT)** in ein natives Binary zu kompilieren. Anstatt Bytecode auf der JVM zur Laufzeit zu JIT-kompilieren, analysiert *native-image* das Programm statisch und erzeugt eine ausführbare Datei (für Linux, Windows oder Mac, je nachdem).

Diese Binary enthält nur das Nötige: Application-Klassen, benötigte Teile der JDK-Bibliotheken und Runtime (z.B. SubstrateVM) und evtl. eingebettete Ressourcen.

Packaging
GraalVM Vorteile

- **Blitzschneller Start:** Native Binaries starten in wenigen **Millisekunden** (kein JVM-Bootstrap, kein JIT-Warmup). In einem Benchmark lag der Startup einer kleinen App bei ~48 ms nativ vs. ~2.1 Sek auf der HotSpot JVM*
- **Geringer Footprint:** Native Images brauchen viel weniger RAM zur Laufzeit. Beispiel: ~125 MB Total (Heap+Native) vs ~400 MB auf der JVM im Durchschnitt, also gut 70% weniger Memory usage bei GraalVM Native in einem Test*. Dadurch können wir mehr Instanzen pro Host starten und sparen Cloud-Kosten
- **Sofortige Performance:** Keine lange Aufwärmphase, das Binary läuft ab Start mit voller Geschwindigkeit, wichtig für sporadisch getriggerte Dienste (FaaS/Serverless)

*https://www.javacodegeeks.com/2025/04/graalvm-vs-openjdk-which-is-faster-in-2024.html

Packaging
GraalVM Nachteile

- **Durchsatz**: In einigen Fällen kann die *Peak Performance* (Durchsatz langer laufender Prozesse) etwas niedriger sein als auf einer optimierten JVM, da GraalVM (native) zur Compile-Zeit optimiert und keine profilgestützte Optimierung während Laufzeit betreibt
- **Geschlossene Welt**: Das Programm muss als *Closed World* kompiliert werden. **Dynamische Java-Features** wie Reflection, dynamisches Klassenladen, Proxy etc. sind nicht automatisch erfasst. Der Native-Image-Builder führt eine **statische Analyse** durch und sieht nur zur Compile-Time *statisch erreichbaren* Code
  Alles was per Reflection zur Laufzeit zugänglich gemacht werden soll, muss explizit im Build bekannt gemacht werden (via **Konfigurations-Dateien** oder Build-Flags) -> Komplexer

*https://www.javacodegeeks.com/2025/04/graalvm-vs-openjdk-which-is-faster-in-2024.html

Packaging
Docker Container

- Docker ist eine Container-Technologie, die Anwendungen inklusive Laufzeitumgebung in standardisierte, isolierte Images verpackt.
- Für Scala- oder Play-Anwendungen nutzen wir das sbt-native-packager Plugin, um automatisch Dockerfiles und Images zu erzeugen.
- Ein Container enthält die App, Abhängigkeiten, Startskripte und ein minimales Betriebssystem – lauffähig auf jeder Docker-fähigen Plattform.
- *"It works on my machine" → "It works everywhere"*

*https://www.javacodegeeks.com/2025/04/graalvm-vs-openjdk-which-is-faster-in-2024.html

innFactory

Packaging
Docker Container Vorteile

- **Portabilität:** Einmal gebaut, überall lauffähig (Linux, Cloud, CI/CD …)
- **Isolation:** Jeder Container ist eine eigenständige, abgeschottete Laufzeitumgebung
- **Reproduzierbarkeit:** Konsistente Builds (keine „funktioniert-nur-lokal“-Probleme)
- **DevOps-freundlich:** Einheitliches Artefakt für Test, Staging, Prod

*https://www.javacodegeeks.com/2025/04/graalvm-vs-openjdk-which-is-faster-in-2024.html

Packaging
Docker Container Nachteile

- **Größe:** JVM-basierte Docker-Images enthalten eine vollständige JRE und sind oft >200 MB groß
- **Startup-Zeit:** Der JVM-Boot dauert Sekunden, für Cold-Start-sensitive Apps (z. B. Serverless) problematisch
- **Dateirechte:** Play benötigt Schreibrechte für RUNNING_PID, was bei Default-Docker-Usern zu Fehlern führen kann
- **Komplexität:** Erfordert Kenntnisse über Dockerfile, Netzwerk, Volume-Mounts, zusätzliche Lernkurve
- **Security:** Base-Images enthalten oft mehr Software als nötig: Angriffsfläche, wenn nicht optimiert (=> Distroless)

*https://www.javacodegeeks.com/2025/04/graalvm-vs-openjdk-which-is-faster-in-2024.html

Übung

Übung
Chimney + Packaging

- Basiert auf Übung von VL06/07 https://github.com/innFactory-Classrooms/afps/tree/main/vl07/play
- Übung:
  - Chimney
    - Alle API Input/Outputs als separate DTOs definieren
    - Transformieren von Slick Row => Domain => DTO bzw. andersherum einbauen, siehe https://chimney.readthedocs.io/
  - Docker Container bauen, siehe sbt-native-packager https://github.com/sbt/sbt-native-packager
    - Sinnvolles Basis-Image verwenden JRE21
    - Play port exposen!
  - Optional / Bonus: Native Image bauen (ebenfalls mit sbt-native-packager möglich)

*https://www.javacodegeeks.com/2025/04/graalvm-vs-openjdk-which-is-faster-in-2024.html